

5. WAP to implement Singly linked list with following operations:
- Create a linked list
 - Deletion of first element, specified element and last element in the list.
 - Display the contents of the linked list.

PSEUDO CODE:

void deleteAtFirst():

struct node *temp

if (head == NULL) then

print "List is empty"

return

END if

temp = head

head = head → next

free temp

END function

void deleteLast():

struct node *temp, *prev

if (head == NULL) then

print "List is empty"

return

END if

if (head → next == NULL) {

free head

head = NULL

return

END if

temp = head;

while (temp → next != NULL) then

prev = temp

temp = temp → next

END While

prev → next = NULL

free temp

END function

void deleteSpecific (int value)

struct node* temp = head, *prev = NULL

if (head == NULL) then

print "List is empty"

return

END if

if (head → data == value) then

deleteFirst()

END if

while (temp != NULL AND temp → data != value) then

prev = temp

temp = temp → next

END while

if (temp == NULL) then

print "value not found"

return

END if

prev → next = temp → next

free temp

END function

12/11/25

CODE:

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

struct node *head = NULL;

void createlist (int n) {
    struct node *newnode, *temp;
    int data, i;
    if (n < 0) {
        printf("n should be greater than 0.");
        return;
    }
    for (i=1; i<=n; i++) {
        newnode = (struct node *) malloc (size of (struct node));
        if (newnode == NULL) {
            printf("Memory allocation failed.");
            return;
        }
        printf("Enter data: ");
        scanf("%d", &data);
        newnode->data = data;
        newnode->next = NULL;
        if (head == NULL) {
            head = newnode;
        }
        else {
            temp->next = newnode;
        }
        temp = newnode;
    }
}

```



```

    }
    printf("Linked list is created.");
}

void deleteFirst() {
    struct node *temp;
    if (head == NULL) {
        printf("List is empty. Create a linked list");
        return;
    }
    temp = head;
    head = temp->next;
    printf("Deleted ele: %d\n", temp->data);
    free(temp);
}

void deleteLast() {
    struct node *temp, *prev;
    if (head == NULL) {
        printf("List is empty. Create a linked list");
        return;
    }
    if (head->next == NULL) {
        printf("Deleted ele: %d\n", head->data);
        free(head);
        head = NULL;
        return;
    }
    temp = head;
    while (temp->next != NULL) {
        prev = temp;
        temp = temp->next;
    }
    prev->next = NULL;
    printf("Deleted ele: %d\n", temp->data);
}

```


free (temp);

```

}
void deleteSpecific (int value) {
    struct node *temp = head, *prev = NULL;
    if (head == NULL) {
        printf ("List is empty. (create a linked list)");
        return;
    }

```

```

    if (head->data == value) {
        deleteAtFirst();
        return;
    }

```

```

    while (temp != NULL && temp->data != value) {
        prev = temp;
        temp = temp->next;
    }

```

```

    if (temp == NULL) {
        printf ("Value not found.");
        return;
    }

```

```

    prev->next = temp->next;
    printf ("Deleted element is %d\n", temp->data);
    free (temp);
}

```

```

void display () {
    struct node *temp = head;
    if (head == NULL) {
        printf ("List is empty.");
    }

```

```

    else {
        printf ("linked list: \n");
        while (temp != NULL) {
            printf ("%d->", temp->data);

```



```

temp = temp->next;
}
}

int main() {
    int ch, n, value;
    do {
        printf("\n 1) Create linked list 2) Delete at first
        3) Delete specific 4) Delete at last
        5) Display list\n");
        printf("\n Enter your choice: ");
        scanf("%d", &ch);
        switch (ch) {
            case 1:
                printf("\n Enter no. of nodes: ");
                scanf("%d", &n);
                createlist(n);
                break;
            case 2:
                deleteAtFirst();
                break;
            case 3:
                deleteSpe
                printf("\n Enter value of node: ");
                scanf("%d", &value);
                deleteSpecific(value);
                break;
            case 4:
                deleteLast();
                break;
            case 5:
                display();
                break;
        }
    } while (ch != 0);
}

```



```

        default: print("Invalid input.");
    }
    while (ch != -1);
}

```

Output:

1) Create linked list 2) Delete at first 3) Delete specific 4) Delete at last 5) Display list.

Enter your choice: 1

Enter no. of nodes: 5

Enter data: 10

Enter data: 20

Enter data: 30

Enter data: 40

Enter data: 50

Linked list is created.

1) Create linked list 2) Delete at first 3) Delete specific 4) Delete at last 5) Display list

Enter your choice: 4

Deleted ele: 50

1) Create linked list 2) Delete at first 3) Delete specific 4) Delete at last 5) Display list

Enter your choice: 3

Enter value: 30

Deleted ele: 30

1) Create linked list 2) Delete at first 3) Delete specific 4) Delete at last 5) Display list

Enter your choice: 2

Deleted ele: 10

- 1) Create linked list 2) Delete at first 3) Delete specific
4) Delete at last 5) Display list.

Enter your choice : 5

Linked list :

~~20~~ → 40 → ~~N~~

17/11/20
Seen